

Snakemake Pipeline - Law of Large Numbers

Reference for the Snakemake rules to be hand-copied onto A4 paper. Range 1.. n is sampled k times; sample mean stored; repeated $reps$ times per k ; one boxplot per k . As k grows, the boxes tighten around the true mean $(n+1)/2$.

Snakefile

```
# Snakemake pipeline: Law of Large Numbers demonstration
#
# For a range 1.. $n$ , draw  $k$  samples and compute the mean.
# Repeat that REPS times so we have a distribution of means per  $k$ .
# Plot all distributions as boxplots – as  $k$  grows, the boxes
# tighten around the true mean  $(n+1)/2$ .
#
# Configure  $n$ ,  $k$  values, and  $reps$  in config.yaml.

configfile: "config.yaml"

N      = config["n"]
K_VALUES = config["k_values"]
REPS    = config["reps"]

rule all:
    input:
        f"results/draws_plot_n{N}.png"

# One rule call per  $k$ . Snakemake will fan these out in parallel.
rule sample:
    output:
        "results/samples_n{n}_k{k}.csv"
    params:
        reps = REPS
    script:
        "scripts/sample.py"

# Collect every  $k$ 's CSV for a given  $n$  and produce the boxplot.
rule plot:
    input:
        expand("results/samples_n{{n}}_k{k}.csv", k=K_VALUES)
    output:
        "results/draws_plot_n{n}.png"
    script:
        "scripts/plot.py"
```

config.yaml

```
# --- Change these to reconfigure the experiment -----
# n      : range to sample from is 1.. $n$  (true mean =  $(n+1)/2$ )
# k_values : sample sizes to test – each becomes one box on the plot
# reps    : how many times to repeat the sampling per  $k$ 
#          (more reps -> smoother boxes)
#
# To add a  $k=5000$  box, just append 5000 to k_values.
# To test the range 1..2000, set n: 2000.
# -----

n: 1000

k_values:
  - 5
  - 10
  - 25
  - 50
  - 100
```

- 200
- 1000
- 2000

reps: 100

scripts/sample.py

```
"""
Draw k integers uniformly from 1..n, take the mean, and repeat REPS times.
Writes a tidy CSV with one row per replicate: columns = k, mean.

Called by Snakemake – `snakemake` global is injected at runtime.
"""

import numpy as np
import pandas as pd

n = int(snakemake.wildcards.n)
k = int(snakemake.wildcards.k)
reps = int(snakemake.params.reps)

# Deterministic per-(n, k) seed so reruns reproduce the same plot.
seed = (n * 1_000_003 + k) & 0xFFFFFFFF
rng = np.random.default_rng(seed)

# Each replicate: draw k samples from 1..n, store the sample mean.
means = rng.integers(1, n + 1, size=(reps, k)).mean(axis=1)

pd.DataFrame({"k": k, "mean": means}).to_csv(snakemake.output[0], index=False)
```

scripts/plot.py

```
"""
Combine every per-k CSV for a given n into one boxplot.
Each box = distribution of sample means across the REPS replicates.
A dashed line at (n+1)/2 marks the true mean.
"""

import pandas as pd
import matplotlib.pyplot as plt

n = int(snakemake.wildcards.n)

# Read every input CSV, then build one list-of-arrays in increasing k order.
frames = [pd.read_csv(f) for f in snakemake.input]
df = pd.concat(frames, ignore_index=True)

k_vals = sorted(df["k"].unique())
data = [df.loc[df["k"] == k, "mean"].values for k in k_vals]

labels = [f"k={k}" for k in k_vals]

fig, ax = plt.subplots(figsize=(11, 6))
# `tick_labels` was renamed from `labels` in matplotlib 3.9 – fall back for older.
try:
    ax.boxplot(data, tick_labels=labels)
except TypeError:
    ax.boxplot(data, labels=labels)

true_mean = (n + 1) / 2
ax.axhline(true_mean, color="red", linestyle="--", alpha=0.6,
            label=f"True mean = {true_mean}")

ax.set_title(f"Testing Draws for {n}")
```

```
ax.set_ylabel("Mean")
ax.set_xlabel("Number of draws")
ax.legend(loc="lower right")

plt.tight_layout()
plt.savefig(snakemake.output[0], dpi=150)
```